

Exoplanet Parameter Estimation Showcase

Reconstructing and Modernizing the Polygence Prototype

Mauricio Tedeschi

July 11, 2026

Abstract

This document accompanies the rebuilt `exoplanet-parameter-estimation-model` project. The original Polygence archive contained a three-script prototype, four real Doppler time-series CSVs, a stellar mass table, and two draw.io design artifacts. The rebuild packages that work into a reproducible uv-managed Python project with a Keplerian radial-velocity model, differential-evolution search, local least-squares refinement, synthetic validation, and physics-based orbit visualization. This report summarizes the archived workflow, explains the updated modeling pipeline, and documents results on both synthetic data and the recovered `star0` observation set.

1 Archived Material

The recovered Polygence folder contributed the following assets:

- legacy scripts: `graphcomparison.py`, `nbodysolver.py`, and `rvdatareader.py`
- a stellar mass catalog for four systems
- four real Doppler-shift datasets: `star0.csv` through `star3.csv`
- two draw.io diagram files labeled “M_p and R space” and “M_p and R space after iteration”

The archive did not include a finished paper or L^AT_EX manuscript, so this report serves as a reconstructed write-up grounded in the recovered code, datasets, and the new implementation.

2 Legacy Prototype

The original project estimated exoplanet parameters by combining three pieces:

1. Doppler-shift CSV ingestion
2. a two-body Euler integrator for star-planet motion
3. a geometric-mean search over planet mass and orbital radius

That approach had educational value and preserved the physical intuition of a star wobbling around the system barycenter. However, it also mixed the expensive numerical integrator directly into the fitting loop, depended on missing local CSV files, and was difficult to reproduce or extend.

3 Modernized Pipeline

The rebuild keeps the same scientific story while separating *modeling*, *optimization*, and *visualization*.

3.1 Doppler Conversion

The archived files store Doppler shift z instead of radial velocity v . The package converts between the two using the relativistic relation

$$z = \sqrt{\frac{1+\beta}{1-\beta}} - 1, \quad \beta = \frac{v}{c}, \quad (1)$$

which can be inverted to recover stellar radial velocity from each observed shift value.

3.2 Keplerian Radial-Velocity Model

For fitting, the rebuilt code uses an analytic single-planet Keplerian model:

$$v_r(t) = \gamma + K [\cos(\omega + \nu(t)) + e \cos(\omega)], \quad (2)$$

where K is the radial-velocity semi-amplitude, P is the orbital period, e is eccentricity, ω is the argument of periastron, t_0 is the periastron time, and γ is the system velocity offset. The true anomaly $\nu(t)$ is recovered by solving Kepler’s equation for the eccentric anomaly and then transforming into orbital phase.

3.3 Optimization Strategy

Instead of evaluating a numerical integrator at every candidate point, the new pipeline:

1. performs a global search with `scipy.optimize.differential_evolution`
2. refines the best candidate with `scipy.optimize.least_squares`
3. derives a representative planet mass and semi-major axis from the fitted semi-amplitude and stellar mass under an edge-on assumption

This split makes fitting fast enough to run as a repeatable example while still preserving a physically interpretable orbit plot.

3.4 Orbit Visualization

After fitting, the project constructs barycentric initial conditions and integrates a two-body orbit using the velocity Verlet method. The resulting orbit panel is not used for optimization; it is a clean post-fit visualization that keeps the original project’s dynamical flavor without the numerical drift of the Euler loop.

4 Datasets

4.1 Synthetic Validation

The repository now ships a synthetic-data generator. A reference system with known K , P , e , stellar mass, and planet mass is sampled at irregular observation times, corrupted with measurement noise, and then passed through the same fitting pipeline. This gives a reproducible benchmark for validating the rebuild.

4.2 Recovered Polygence Data

The archived real observations are now versioned under `data/polygence/`. The stellar mass catalog is:

<code>star0:</code>	<code>1.11 $M_{\odot}$</code>	<code>star2:</code>	<code>2.30 $M_{\odot}$</code>
<code>star1:</code>	<code>1.39 $M_{\odot}$</code>	<code>star3:</code>	<code>1.34 $M_{\odot}$</code>

The rebuilt example script currently demonstrates `star0`, matching the legacy prototype’s original short test case and providing a compact real-data sanity check for the new package.

5 Results

5.1 Synthetic Benchmark

For the synthetic showcase system, the recovered fit is close to the planted truth:

- truth: $P = 147.5$ days, $K = 30.5$ m/s, $M_p = 0.82 M_J$, $e = 0.19$
- recovered: $P = 147.16$ days, $K = 31.35$ m/s, $M_p = 0.836 M_J$, $e = 0.221$
- residual RMS: 2.11 m/s

Exoplanet Parameter Estimation from Radial Velocity

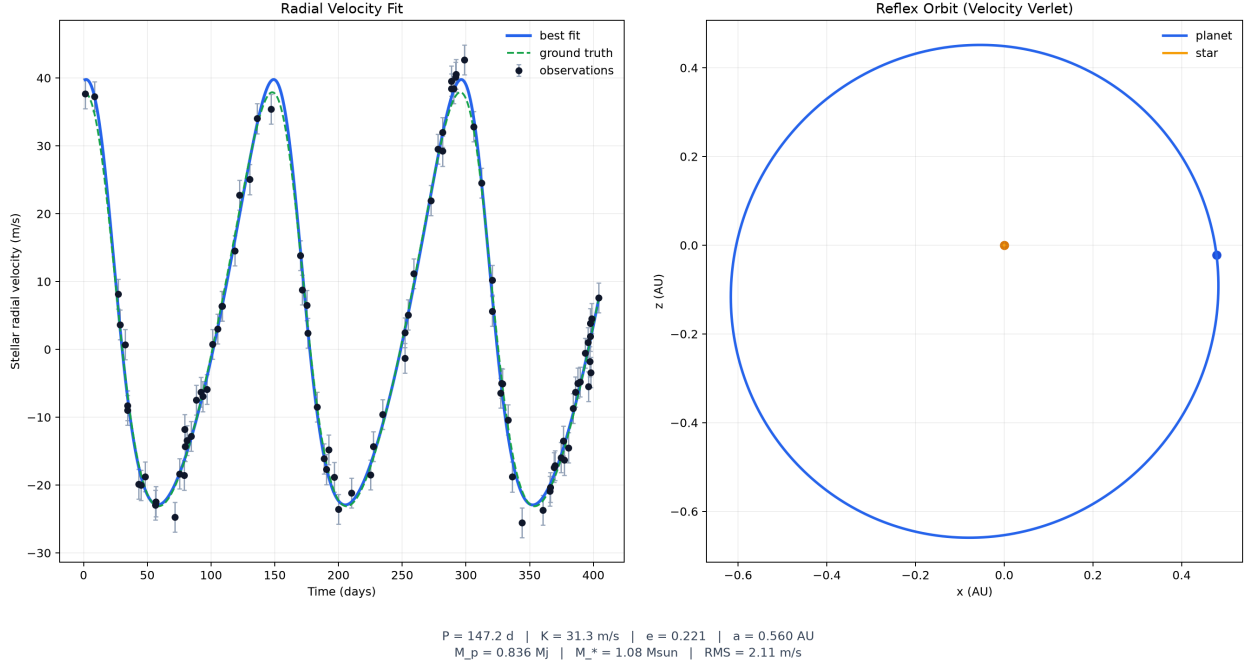


Figure 1: Synthetic validation example included in the rebuilt repository. The left panel shows noisy radial-velocity samples with the recovered Keplerian fit; the right panel shows the corresponding barycentric orbit rendered with the velocity Verlet integrator.

5.2 Recovered Polygence star0 Fit

Running the new real-data example on `star0` yields:

- stellar mass: $1.11 M_{\odot}$
- fitted period: 4.269 days
- fitted semi-amplitude: 56.694 m/s
- fitted eccentricity: 0.002
- derived planet mass: $0.485 M_J$
- residual RMS: 5.23 m/s

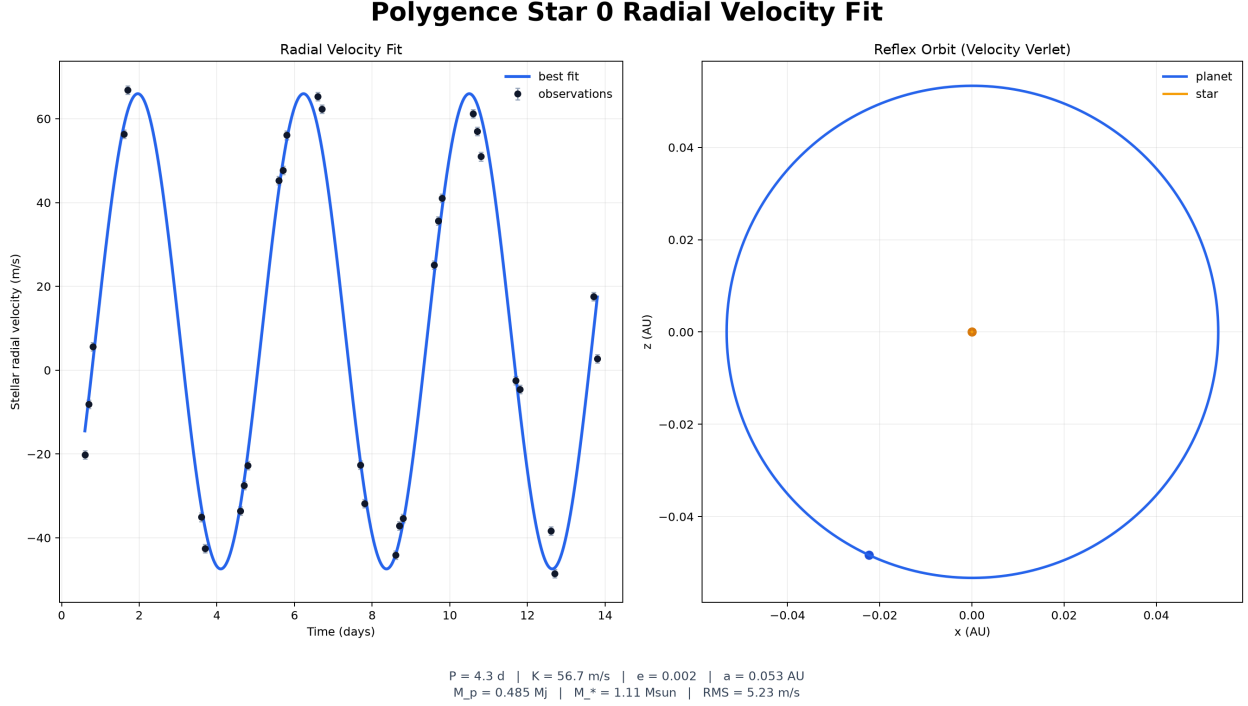


Figure 2: Recovered fit for the archived `star0` dataset. The nearly circular best fit and short orbital period make this a clean real-data example for the rebuilt package.

6 Repository Additions

The rewrite now includes:

- packaged source code under `src/exoplanet_est/`
- synthetic and real-data example scripts
- archived Polygence CSVs under `data/polygence/`
- documentation figures under `docs/figures/`
- this $\text{\LaTeX}$ report as a companion artifact

7 Limitations and Next Steps

The current package intentionally focuses on the clean single-planet case. A few natural extensions would be:

- support explicit measurement uncertainties and instrument offsets when available
- add periodogram-based initialization for the longest archival datasets
- extend the model to multiple planets or Bayesian posterior estimation
- convert the draw.io design artifacts into polished report figures if a future revision needs more visual explanation of the original search space

8 Reproduction

The core commands are:

```
uv sync
MPLBACKEND=Agg uv run python examples/fit_synthetic.py --save outputs/rv_fit_preview.png
MPLBACKEND=Agg uv run python examples/fit_polygence_star.py --star-index 0 \
    --save outputs/polygence_star0_fit.png
latexmk -pdf -cd docs/exoplanet_showcase_report.tex
```

Together, these commands reproduce the synthetic benchmark, the archived `star0` fit, and the compiled report.